

MARAN PMS — Módulo Restaurant

PROMPT: Fixes y mejoras — 12 puntos

FIX 1 — Reservas: label “Nombre del huésped” → “Nombre del huésped / cliente”

En el form de nueva reserva de restaurant (`isReservationDialogOpen`), el campo `guestName` dice “Nombre del huésped *”. Cambiar el label:

```
// Antes:
<FormLabel>Nombre del huesped *</FormLabel>
<Input {...field} placeholder="Nombre completo" />

// Después:
<FormLabel>Nombre del huésped / cliente *</FormLabel>
<Input {...field} placeholder="Nombre completo" />
```

FIX 2 — Reservas: estado inicial directo a “confirmada”, eliminar paso “pendiente”

Al crear una reserva de restaurant, el status debe ser `"confirmed"` directamente, sin pasar por `"pending"` .

En el backend — `POST /api/restaurant/table-reservations` :

```
// Asegurarse de que el status por defecto sea 'confirmed':
const newReservation = await storage.createTableReservation({
  ...req.body,
  status: req.body.status || "confirmed", // default confirmed, no pending
});
```

En el frontend — al enviar el form:

En el `createReservationMutation`, agregar `status: "confirmed"` explícitamente en el payload:

```
mutationFn: async (data: ReservationFormValues) => {
  const res = await apiRequest("POST", "/api/restaurant/table-reservations", {
    ...data,
    status: "confirmed", // ← siempre confirmada al crear
  });
  return res.json();
},
```

Ocultar el estado “pending” de los filtros y badges:

Si hay un filtro o selector de estados que muestra “Pendiente”, quitarlo de las opciones visibles para reservas de restaurant. El estado `"pending"` puede quedar en el tipo TypeScript por compatibilidad pero no debe ser seleccionable ni crearse por defecto.

FIX 3 — Plano de mesas: colorear mesas con reserva en azul

Actualmente el plano solo colorea verde (libre) y rojo (ocupada). Las mesas con reserva para hoy deben mostrarse en azul.

Lógica:

En el render del plano de mesas, antes de aplicar `tableStatusColors[table.status]`, verificar si la mesa tiene una reserva confirmada para hoy:

```
// Dentro del map de areaTables en el plano:
const today = new Date().toISOString().split("T")[0];
const hasReservationToday = reservations.some(
  (r) =>
    r.tableId === table.id &&
    r.reservationDate === today &&
    (r.status === "confirmed" || r.status === "pending")
);

// Determinar color efectivo:
const effectiveStatus = table.status === "available" && hasReservationToday
  ? "reserved"
  : table.status;

// Usar effectiveStatus en lugar de table.status:
className={`... ${tableStatusColors[effectiveStatus]} ...`}
```

El objeto `tableStatusColors` ya tiene la clase para `reserved` :

```
reserved: "bg-blue-500/20 text-blue-700 dark:text-blue-400 border-blue-500/40",
```

Agregar la referencia visual en la leyenda del plano:

La leyenda ya existe (muestra verde=libre, rojo=ocupada). Verificar que

`tableStatusLabels` **incluya `reserved` con el label correcto:**

```
const tableStatusLabels: Record<string, string> = {
  available: "Libre",
  occupied: "Ocupada",
  reserved: "Reservada",    // ← agregar si no existe
  cleaning: "Limpieza",
  blocked: "Bloqueada",
};
```

FIX 4 — Sectores: habilitar edición de nombre + reordenar (Moneda primero)

A — Agregar botón de editar nombre en la lista de sectores:

En la sección de configuración de áreas del restaurant, agregar un botón de edición inline por cada área:

```
{areas.map((area) => (
  <div key={area.id} className="flex items-center justify-between p-3 border roun
    {editingAreaId === area.id ? (
      <div className="flex items-center gap-2 flex-1">
        <Input
          value={editingAreaName}
          onChange={ (e) => setEditingAreaName(e.target.value)}
          className="h-8"
          autoFocus
        />
        <Button size="sm" onClick={() => handleSaveAreaName(area.id)}>
          <Check className="h-4 w-4" />
        </Button>
        <Button size="sm" variant="outline" onClick={() => setEditingAreaId(null)}
          <X className="h-4 w-4" />

```

```

        </Button>
      </div>
    ) : (
      <>
        <span className="font-medium">{area.name}</span>
        <Button
          size="sm"
          variant="ghost"
          onClick={() => {
            setEditingAreaId(area.id);
            setEditingAreaName(area.name);
          }}
        >
          <Pencil className="h-4 w-4" />
        </Button>
      </>
    )}
  </div>
)}

```

Agregar estados: `const [editingAreaId, setEditingAreaId] = useState<string | null>(null)` **y** `const [editingAreaName, setEditingAreaName] = useState("")`.

Handler:

```

const handleSaveAreaName = async (areaId: string) => {
  await apiRequest("PATCH", `/api/restaurant/areas/${areaId}`, { name: editingAreaName });
  queryClient.invalidateQueries({ queryKey: ["/api/restaurant/areas"] });
  setEditingAreaId(null);
};

```

B — Reordenar sectores: “Sector Moneda” primero

En la DB, el sector principal (el que tiene las mesas 1-14, llamado “area1” o similar) debe tener `displayOrder = 0`. El sector “Gourmet/Resto-bar” (mesas 15-29) debe tener `displayOrder = 1`.

Agregar al inicio del servidor (migración de orden):

```

await db.execute(sql`
  UPDATE restaurant_areas SET display_order = 0 WHERE id = 'area1'
`);
await db.execute(sql`
  UPDATE restaurant_areas SET display_order = 1 WHERE id = 'area2'
`);

```

Y ordenar en el frontend/backend al hacer GET:

```
SELECT * FROM restaurant_areas ORDER BY display_order ASC, name ASC
```

FIX 5 — Cobro a habitación: bloquear métodos de pago no válidos

Cuando el método de pago seleccionado es `"cuenta_habitacion"`, solo deben estar disponibles:

- `cuenta_habitacion` — Cargo a habitación
- `voucher` — Voucher (no fiscal) como tipo de comprobante

Tanto en el cierre normal de mesa como en el cierre por división (splits), aplicar el mismo filtro.

Este comportamiento **ya existe para pedidos sin mesa** (`isTableless`). Extenderlo para cuando el usuario selecciona `"cuenta_habitacion"` en cualquier tipo de cierre:

```
// En el selector de forma de pago del cierre:
// Cuando se selecciona cuenta_habitacion → forzar receiptType a "voucher"
// y NO mostrar los otros métodos de pago simultáneamente en splits

// En el cierre por división (splits):
const splitMethodsForRoom = {
  cuenta_habitacion: "Cargo a habitación",
  // solo este método cuando se cobra a habitación
};
```

En el cierre por división, si una parte se cobra a habitación, solo mostrar el selector de habitación para esa parte y forzar comprobante a voucher.

Filtro del selector de habitación — mostrar solo ocupadas y buscar por número o nombre:

Actualmente ya usa `inHouseRooms` (que filtra `status === "occupied"` y `checked_in`). Agregar un campo de búsqueda sobre ese listado:

```
{closePaymentMethod === "cuenta_habitacion" && (
  <div className="space-y-2">
```

```

<Label>Habitación</Label>
<Input
  placeholder="Buscar por número o nombre..."
  value={roomSearchFilter}
  onChange={(e) => setRoomSearchFilter(e.target.value)}
  className="mb-2"
/>
<Select value={closeRoomId} onValueChange={setCloseRoomId}>
  <SelectTrigger><SelectValue placeholder="Seleccionar habitación" /></Select
  <SelectContent>
    {inHouseRooms
      .filter((r) =>
        roomSearchFilter === "" ||
        r.roomNumber.includes(roomSearchFilter) ||
        r.guestName.toLowerCase().includes(roomSearchFilter.toLowerCase())
      )
      .map((r) => (
        <SelectItem key={r.roomId} value={r.reservationId}>
          {r.roomNumber} - {r.guestName}
        </SelectItem>
      ))}
  </SelectContent>
</Select>
</div>
)}

```

Agregar estado: `const [roomSearchFilter, setRoomSearchFilter] = useState("") .`

Limpiar el filtro al cerrar el dialog: `setRoomSearchFilter("") .`

FIX 6 — Ítems de la comanda: permitir editar curso antes de que salga a cocina

Actualmente la única forma de cambiar el curso de un ítem ya comandado es borrarlo y volver a ingresarlo. Agregar la posibilidad de editar el curso mientras el ítem no haya salido a cocina (es decir, mientras el ítem tenga `status !== "sent"` o el estado equivalente a "enviado a cocina").

En la vista de ítems de la comanda (lista de lo comandado):

Agregar un botón de edición rápida de curso por ítem:

```

{getOrderItems().map((item) => (
  <div key={item.id} className="flex items-center justify-between p-2 border roun

```

```

<div>
  <span>{item.menuItem?.name || item.customItemName}</span>
  <span className="text-xs text-muted-foreground ml-2">
    {item.course ? `Curso ${item.course}` : "Sin curso"}
  </span>
</div>
<div className="flex items-center gap-1">
  {/* Solo mostrar si no fue enviado a cocina */}
  {!item.sentAt && (
    <div className="flex gap-1">
      {[1, 2, 3].map((c) => (
        <button
          key={c}
          className={`w-6 h-6 text-xs rounded ${item.course === c ? "bg-prima"
            onClick={() => updateItemCourseMutation.mutate({ itemId: item.id, c
          >
            {c}
          </button>
        )}}
      </div>
    )}
  </div>
</div>
)}}

```

Mutation para actualizar curso:

```

const updateItemCourseMutation = useMutation({
  mutationFn: ({ itemId, course }: { itemId: string; course: number }) =>
    apiRequest("PATCH", `/api/restaurant/order-items/${itemId}`, { course }),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/restaurant/orders"] });
  },
});

```

Endpoint backend:

```

app.patch("/api/restaurant/order-items/:id", requireAuth, async (req, res) => {
  try {
    const { course, notes } = req.body;
    const updated = await storage.updateOrderItem(req.params.id, { course, notes
    res.json(updated);
  } catch (e: any) {
    res.status(500).json({ error: e.message });
  }
});

```

```
}  
});
```

FIX 7 — Metodología de comanda: sale completa desde el inicio

Cambio conceptual importante: La comanda debe salir completa a cocina desde el primer momento (entrada + principal + postre juntos). Los mozos luego dan la señal "Sale" cuando el comensal terminó la entrada y está listo para el principal.

Cambiar el comportamiento del botón "Siguiente Curso":

- **Antes:** "Siguiente curso" implica que los platos van saliendo por etapas a cocina.
- **Después:** Renombrar "Siguiente Curso" a **"Sale → Principal"** (o simplemente **"Sale"**). Este botón solo emite una señal/notificación visual, no retiene ni envía nuevos ítems.

```
// Cambiar el label del botón:  
<Button onClick={handleNextCourse}>  
  <ChefHat className="h-4 w-4 mr-2" />  
  Sale — Despachar Principal  
</Button>
```

La comanda ya fue enviada completa al abrirse. El botón "Sale" simplemente marca visualmente que el principal está siendo despachado, para que la comandera (cuando se conecte) lo sepa.

FIX 8 — Bebidas: sin curso fijo, no tomar el curso del último ítem

Las bebidas deben tener `course = null` siempre, independientemente del orden en que se ingresen.

En el selector de curso del `pendingItem`:

La lógica ya detecta si es bebida por categoría. Asegurarse de que al agregar una bebida, el `itemCourse` se fuerce a `null` o `0`:


```
// En el handler de addItem cuando isPending:
const isBeverage = beverageCategories.some(bc =>
  cat?.name.toLowerCase().includes(bc)
);

await apiRequest("POST", `/api/restaurant/orders/${currentOrder.id}/items`, {
  menuItemId: pendingItem.id,
  quantity: itemQuantity,
  course: isBeverage ? null : itemCourse, // ← null siempre para bebidas
  unitPrice: pendingItem.price,
  customItemName: isEditableItem ? customItemName : null,
  customItemPrice: isEditableItem ? customItemPrice : null,
});
```

Actualmente el código detecta la bebida y muestra “sin curso” en el UI, pero al guardar puede estar enviando el `itemCourse` del estado (que podría ser 1, 2 o 3 del ítem anterior). Forzar `null` explícitamente en el payload.

FIX 9 — Delivery: solo voucher, sin selector de método de pago

El delivery ya viene cobrado por la app externa (PedidosYa, Rappi, etc.). El sistema solo registra el voucher interno como constancia. **No debe mostrar ningún selector de método de pago.**

```
const isDelivery = updOrder?.areaId === "area-delivery";
```

Cuando `isDelivery === true`:

- Tipo de comprobante fijo: **Voucher (No Fiscal)** — no editable, mostrar como texto, no como Select.
- **Ocultar completamente** el selector de forma de pago.
- El botón de confirmar cierre dice **“Emitir Voucher”** en lugar de “Confirmar Cierre”.

```
{isDelivery ? (
  <div className="p-3 rounded-md bg-muted/50 text-sm text-muted-foreground text-c
    <Receipt className="h-4 w-4 mx-auto mb-1" />
    Delivery — cobro gestionado por la app.<br />
    <span className="text-xs">Se emitirá Voucher no fiscal automáticamente.</span>
  </div>
```

```
) : (  
  // ... selector normal de comprobante y método de pago  
})
```

En el backend, al recibir el cierre de un pedido de Delivery, forzar:

```
receiptType: "voucher",  
paymentMethod: "delivery_app", // nuevo valor, o simplemente "voucher"
```

FIX 10 — Contador de mesas y capacidad: sincronizar con el layout real

En el header de cada área del plano se muestra `{areaTables.length}` mesas | Capacidad: `{area.capacity}`. El problema es que `area.capacity` es un campo fijo en la DB que no se actualiza cuando se editan las mesas.

Calcular la capacidad en tiempo real a partir de las mesas:

```
// En lugar de area.capacity, calcular desde las mesas:  
const realCapacity = areaTables.reduce((sum, t) => sum + (t.capacity || 0), 0);  
  
// Render:  
<span className="text-sm text-muted-foreground">  
  {areaTables.length} mesa{areaTables.length !== 1 ? "s" : ""} | Capacidad: {real  
</span>
```

Esto hace que el número sea siempre correcto sin depender del campo `capacity` del área.

FIX 11 — Solárium y Spa: unificar o permitir eliminar áreas

Los sectores "Solárium" y "Spa" (`area-solarium` y `area-spa`) son redundantes: en la práctica solo el Solárium tiene mesas activas.

Opción A — Desactivar el área Spa:

```
await db.execute(sql`  
  UPDATE restaurant_areas SET is_active = false WHERE id = 'area-spa'
```

```
`);
```

Y en el frontend filtrar las áreas inactivas del plano y del selector:

```
const activeAreas = areas.filter(a => a.isActive !== "false");
```

Opción B — Permitir eliminar áreas vacías desde la UI:

En la lista de sectores de configuración, mostrar un botón de eliminar solo si el área no tiene mesas:

```
{areaTables.length === 0 && (
  <Button
    size="sm"
    variant="ghost"
    className="text-destructive"
    onClick={() => deleteAreaMutation.mutate(area.id)}
  >
    <Trash2 className="h-4 w-4" />
  </Button>
)}
```

Aplicar la Opción A como default (desactivar Spa), y agregar la Opción B para que el usuario pueda gestionar áreas desde la UI en el futuro.

FIX 12 — Unificar “Cerrar Mesa” y “Cerrar Cuenta”

Actualmente existen dos puntos de entrada para cerrar:

- **“Cerrar Cuenta”** — botón dentro de la comanda (abre `isCloseDialogOpen`)
- **“Cerrar Mesa”** — sección en el folio lateral de la mesa

Ambos desencadenan el mismo flujo de cierre. Unificar dejando solo **“Cerrar Cuenta”** como acción principal, y eliminar la sección “Cerrar Mesa” del folio lateral que duplica la funcionalidad.

Si el folio lateral tiene información adicional útil (historial de pagos parciales, etc.), mantener esa información pero quitar el botón de cierre duplicado y reemplazarlo por un botón que lleve al dialog de cierre principal:

```
// En el folio lateral, reemplazar la sección "Cerrar Mesa" por:
```

```

<Button
  variant="destructive"
  className="w-full"
  onClick={() => { setIsCloseDialogOpen(true); }}
>
  <Receipt className="h-4 w-4 mr-2" />
  Cerrar Cuenta
</Button>

```

FIX 13 — Comanda: campo de búsqueda de ítems por texto

Actualmente los ítems solo se pueden agregar haciendo clic en botones agrupados por categoría. Agregar un campo de búsqueda por texto sobre la grilla de ítems, similar al sistema anterior:

```

// Agregar arriba de los botones de categoría en orderView === "menu":
<div className="relative">
  <Search className="absolute left-3 top-2.5 h-4 w-4 text-muted-foreground" />
  <Input
    placeholder="Buscar ítem..."
    value={menuSearchQuery}
    onChange={(e) => {
      setMenuSearchQuery(e.target.value);
      if (e.target.value) setSelectedCategory(null); // al buscar, quitar filtro
    }}
    className="pl-9"
    data-testid="input-menu-search"
  />
</div>

```

Agregar estado: `const [menuSearchQuery, setMenuSearchQuery] = useState("") .`

Modificar el filtro de ítems:

```

const filteredMenuItems = menuItems.filter((item) => {
  if (item.isAvailable === "false") return false;
  if (menuSearchQuery) {
    return item.name.toLowerCase().includes(menuSearchQuery.toLowerCase());
  }
  return selectedCategory ? item.categoryId === selectedCategory : false;
});

```

Limpiar el search al cerrar la comanda: `setMenuSearchQuery("") .`

FIX 14 — Plano de mesas: cronómetro y botón “Sale” desde la mesa

En el plano de mesas, cada mesa ocupada debería mostrar:

- El tiempo transcurrido desde que se abrió la mesa
- Un indicador visual del curso actual (entrada / principal / postre)
- Un botón “Sale” directamente en la mesa para no tener que entrar a la comanda

```
// Dentro del render de la mesa ocupada en el plano:
{table.status === "occupied" && () => {
  const tableOrder = activeOrders.find(o => o.tableId === table.id);
  if (!tableOrder) return null;

  const openedAt = tableOrder.createdAt ? new Date(tableOrder.createdAt) : null;
  const minutesOpen = openedAt
    ? Math.floor((Date.now() - openedAt.getTime()) / 60000)
    : null;

  return (
    <>
    {tableOrder.waiterName && (
      <span className="text-[9px] truncate max-w-full opacity-80">
        {tableOrder.waiterName}
      </span>
    )}
    {minutesOpen !== null && (
      <span className="text-[9px] opacity-70">
        ⌚ {minutesOpen}min
      </span>
    )}
    {/* Botón Sale — pequeño, visible sobre la mesa */}
    <button
      className="text-[8px] bg-orange-500 text-white rounded px-1 mt-0.5"
      onClick={(e) => {
        e.stopPropagation(); // no abrir la comanda
        handleNextCourseForTable(tableOrder.id);
      }}
      title="Despachar siguiente curso"
    >
      SALE
    </button>
  </>
)
```

```
);  
}) () }
```

La función `handleNextCourseForTable` llama al mismo endpoint que el botón "Siguiente Curso" dentro de la comanda.

NOTAS DE IMPLEMENTACIÓN

1. Los fixes **1, 2, 3, 4, 5, 9, 10, 11, 12, 13** son independientes entre sí y se pueden implementar en cualquier orden.
2. Los fixes **7 y 8** están relacionados (metodología de comanda y bebidas sin curso). Implementarlos juntos.
3. Los fixes **6 y 14** son mejoras UX avanzadas. Implementar después de los demás.
4. El fix **4B** (reordenar sectores) requiere saber qué ID tiene el "Sector Moneda" en la DB real. Si es `"area1"`, usar ese ID. Si no, buscar el área cuyas mesas van de la 1 a la 14.
5. Para el fix **14** (cronómetro), el `createdAt` del order debe existir en la tabla `restaurant_orders`. Verificar que el campo esté presente antes de implementar el cronómetro.